

机器学习高级话题

目 录

1 引言：通向“经验时代”	3
2 强化学习基础框架	3
2.1 RL 的基本循环	3
2.2 基于模型 vs 无模型强化学习	3
2.3 高性能模型学习的组成	4
2.4 经验回放缓存	4
2.5 深度 Q 学习 (DQN)	4
3 自举误差与模型接力方法	4
3.1 自举误差的来源	4
3.2 先监督后强化的接力架构	5
3.3 上下文预测模型	5
3.4 观测聚类与原型映射	5
3.5 接力效果	6
4 基于历史样本建模的多智能博弈探索	6
4.1 颤抖手与 epsilon-greedy	6
4.2 探索时机的重要性	6
4.3 蒙特卡洛轨迹树 (MCT2) 规划	6
4.4 稳定前缀策略	6
4.5 策略训练与 Q 值聚合	7
5 自适应 TD(λ) 计算方法	7
5.1 TD(λ) 的平衡难题	7
5.2 相关工作回顾	7
5.3 离策略不稳定的根源	8
5.4 ATD(λ) 方法	8
5.5 同策略程度的计算	8
5.6 ATD 方法的增强效果	9
6 大模型与强化学习	9
6.1 为什么要用大模型做决策	9
6.2 行为树代码生成: Planner-Coder-Critic	9
6.3 实战案例: Stalker 对抗 Spine Crawler	9
6.4 行为树生成能力蒸馏与提升	11
6.5 GRPO 与思考长度	11
6.6 行为树策略辅助强化学习训练	11
6.7 SMAC-Hard	11

7 并行加速：PaMoRL	12
7.1 MBRL 的计算代价	12
7.2 并行扫描（Parallel Scan）	12
7.3 PaMoRL 框架	12
7.4 Linear Attention 变体	13
7.5 PaMoRL 的效果	13
8 具身智能	13
8.1 核心问题	13
8.2 仿真平台	13
8.3 Real2Sim / Sim2Real 闭环	14
8.4 UniHSI	14
9 本章你将学会	14
10 要点速查	15
11 小结	15

1 引言：通向“经验时代”

2024 年图灵奖得主 **Richard Sutton**（理查德·萨顿）现任阿尔伯塔大学计算机科学教授、阿尔伯塔机器智能研究所研究员兼首席科学顾问，他在现代计算强化学习领域做出多项重要贡献，包括时序差分学习和策略梯度方法。Sutton 指出：“从 70 年人工智能研究中可以汲取的最大教训是，长期来看真正重要的只有对计算资源的运用。”由于摩尔定律，依赖人类知识的方法往往以降低计算通用性为代价来增加系统复杂度，只能寻求短期内的显著改进。

当前 AI 主要依赖大量人类生成的数据进行训练，这种方法无法让 AI 学到超越人类理解的新见解，尤其在数学、编程和科学等关键领域。因此，需要一种新设想或方法框架，让 AI 智能体通过与环境交互产生的数据，即**经验**（Experience），来学习，实现超越人类的智能。Sutton 在“Welcome to the Era of Experience”一文中将其称为“经验时代”。

直觉 你可以把监督学习想象成“学生看老师的笔记学知识”，而强化学习是“学生自己动手实验，从结果中总结规律”。前者受限于老师（人类数据）的水平，后者却可能发现老师从未想到的解法。

强化学习（Reinforcement Learning, RL）正是这种“智能体通过与环境交互，学习到具体策略，进而最大化累计奖励”的范式。在多智能体场景下，多个智能体各自与环境交互，学习策略以最大化全局累计奖励。它解决的是从“被动学习知识”到“主动积累经验”的根本问题。

本章围绕六个核心主题展开：基于经验缓存样本监督的模型接力方法、基于历史样本建模的多智能博弈探索、自适应 $TD(\lambda)$ 计算方法、行为树代码生成策略优化、并行化基于模型的强化学习（PaMoRL），以及具身智能。

2 强化学习基础框架

2.1 RL 的基本循环

智能体（Agent）在环境（Environment）中观察状态 s ，采取动作 a ，获得奖励 r ，并转移到新状态 s' 。目标是学习策略 $\pi(a|s)$ 以最大化累计奖励 $\sum \gamma^t r_t$ ，其中 $\gamma \in [0, 1)$ 是折扣因子。

直觉 折扣因子的含义是：眼前的奖励比未来的奖励更“值钱”。如果 $\gamma = 0$ ，智能体只关心下一步的奖励，目光极其短浅；如果 $\gamma \rightarrow 1$ ，智能体关心所有未来奖励的总和，目光长远。实际中常取 $\gamma = 0.99$ 左右。

2.2 基于模型 vs 无模型强化学习

- **基于模型的强化学习**（Model-Based RL, MBRL）：先学习环境模型，再基于模型训练策略。可以直接用 on-policy 强化学习算法更新策略，可以选择不需要更多真实交互数据，在实验上始终比无模型方法具备更高的采样效率。但是受限于模型预测的**累积误差**。
- **无模型强化学习**（Model-Free RL, MFRL）：直接从经验中学习。具有最佳渐近性能，非常适合具有大数据的深度学习架构，但是 off-policy 方法存在不稳定性，且**样本效率极低**，需要大量的训练数据。

两者的权衡是强化学习的核心命题：MBRL 高效但有累积误差，MFRL 低效但渐近好。

2.3 高性能模型学习的组成

MBRL 之所以高效，源于两大支柱：

1. **强大的世界模型**：模型架构包括 RNN、CNN、Transformer 等；模型集成方法包括 MoE、Disagreement、Population 等；辅助任务包括 VAE、SPR、Prototype、Simsam 等。
2. **规划与搜索机制**：多步价值估计（Multi-step Value Estimation）、资格迹（Eligibility Trace，如 VTrace、ReTrace、GAE 等）以及 Multi-step Return。

2.4 经验回放缓存

经验回放缓存（Replay Buffer）存储智能体与环境交互得到的历史经验 $\{s, a, r, s'\}$ ，用于采样训练当前策略。智能体从经验缓存中采样数据来训练策略，这使得历史交互数据可以被反复利用，而非只用一次就丢弃。

经验回放缓存本质上是在回答一个问题：“我当初是怎样做的决策？现在应该怎样做决策？”通过反复审视过去的经验，智能体可以不断改进策略。

2.5 深度 Q 学习（DQN）

经典 **深度 Q 学习**（Deep Q-Network, DQN）通过目标网络（Target Network）计算 Q 值的回归目标。完整流程如下：

1. 基于当前策略采取行为 a_t ，观测 s_t, a_t, r_t, s_{t+1} 并加入到缓存 B ；
2. 均匀从缓存 B 中采样得到小组数据 $\{s_j, a_j, r_j, s_{j'}\}$ ；
3. 利用目标 Q 网络计算回归目标 $y_j \leftarrow r_j + \gamma \max_{a'} Q_{\text{target}}(s_{j'}, a')$ ；
4. 用梯度下降更新当前网络参数 ω ；
5. 更新目标网络参数 ω^- 。

例 假设 $\gamma = 0.9$ ，在某个状态下智能体采取动作后获得即时奖励 $r = 2$ ，目标网络估计下一状态的最大 Q 值为 $Q_{\text{target}}(s', a') = 5$ 。则回归目标 $y = r + \gamma \max_{a'} Q_{\text{target}}(s', a') = 2 + 0.9 \times 5 = 6.5$ 。当前网络若输出 $Q(s, a) = 4$ ，则误差为 $4 - 6.5 = -2.5$ ，参数朝减小此误差的方向更新。目标网络定期同步，避免“追着自己的影子跑”。

3 自举误差与模型接力方法

3.1 自举误差的来源

自举误差（Bootstrapping Error）是指智能体输出的预期状态-行为价值，与环境真实采样得到的累计奖励之间的差异。

直觉 想象你用一个不太准的天气预报来预测明天的天气，再用明天的预测来推后天的，如此传递下去。每一步的小误差都会在链式传递中被放大，越往后的预测越不可信。自举误差在 MDP 决策树中也是如此。

自举误差有两个关键特性：

- 自举误差随 MDP 树向根节点回传，层层累积放大；
- 智能体数量增大，自举误差显著增大，导致早期决策不可信。

例 在多智能体强化学习的初期阶段，Q 值学习的随机性较高且未收敛，因此每个决策步骤中的自举误差很大。假设 1 个智能体时单步误差为 ϵ ，3 个智能体时累积误差可能达到 3ϵ 量级，5 个智能体时更大。这导致早期决策完全不可信。

3.2 先监督后强化的接力架构

既然早期决策不可信，论文提出了“先监督学习后强化学习”的**接力决策架构**（Relay Architecture）。核心思路是：先用监督学习给出一个好的策略起点，再用基线多智能体强化学习算法接力训练。

具体做法是将经验缓存中的轨迹建模成**蒙特卡洛轨迹树**（Monte Carlo Trajectory Tree）的形式，通过 UCB 计算采样出最优轨迹，并以**行为克隆**（Behavioral Cloning）的模式学习轨迹进行训练。在决策时先用监督学习策略，再用基线多智能体强化学习算法接力训练。

3.3 上下文预测模型

上下文预测模型（Context Prediction Model）是接力架构的核心组件。给定一条轨迹 $s_t, a_t, o_t, s_{t+1}, a_{t+1}, o_{t+1}, \dots$ ，该模型以当前观测 o_t 为输入，同时输出两个目标：

- **动作预测** a_t ：作为分类任务 $\Phi_{\text{action}}(a|o_t, \omega)$ ，采用交叉熵损失函数；
- **下一时刻观测预测** o_{t+1} ：作为回归任务 $\Phi_{\text{obs}}(o_{t+1}|o_t, \omega)$ ，采用均方误差损失函数。

直觉 上下文预测器就像一个“双任务大脑”：一方面要决定“现在该做什么动作”（分类），另一方面要预测“做完这个动作后会看到什么”（回归）。如果它的观测预测与真实推演后的观测相似，说明它对当前局面理解准确，可以继续由它决策；如果不相似，说明局面已超出它的理解范围，交由强化学习模型接手。

3.4 观测聚类与原型映射

由于状态和观测是连续值，直接建立树的节点是不现实的，同时上下文预测值与真实值对比时几乎无法保证完全相同。因此观测值需要被映射到具备一定误差阈值的**原型**（Prototype）。从观测到原型的映射可以采用 EM 算法完成聚类。

蒙特卡洛轨迹树的生成流程为：

1. 将经验缓存中的轨迹数据进行聚类形成原型；
2. 将轨迹数据转换为原型数据；
3. 将一定量的原型轨迹建模为蒙特卡洛轨迹树；
4. 从轨迹树中从根节点到叶子节点进行选择，选出当前最优原型轨迹路径。

这里用原型而非原始观测来构建树的节点，本质上是对连续状态空间做离散化近似，类似于将无限细分的颜色归并为有限的色板。

3.5 接力效果

在多智能体粒子世界环境和 SMAC 环境模拟中，该接力方法在算法收敛速度和表现上都有较大的提升。随着训练的进行，接力比率增大，即上下文预测器决策更多的时间步，说明监督学习策略逐渐被信任。该方法对极大数量智能体环境的支持，以及对单智能体强化学习的兼容性也得到了验证。

4 基于历史样本建模的多智能博弈探索

4.1 颤抖手与 epsilon-greedy

epsilon-greedy 探索方法可以被视为颤抖手决策问题 (Trembling Hand Perfect Equilibrium)：以概率 p 进行探索，随机选择一个行为；以概率 $1 - p$ 进行利用，选择价值最大的行为。颤抖手均衡的直觉是，以较大概率完成最大价值决策，但以小概率出现“手滑”，选错其他选项。

例 假定 $\varepsilon = 0.2$ ，在某个博弈中颤抖手均衡解为 (T, L)，然而最优解是 (B, R)。这说明纯粹的 epsilon-greedy 探索可能引导智能体走向次优均衡。

4.2 探索时机的重要性

给定一个策略：两个智能体朝向终点运动，终点附近有墙阻挡。

- 如果不进行探索，智能体被墙挡住，陷入局部最优；
- 如果在全路径都进行探索，智能体可能会进行很多不必要的探索过程；
- 如果智能体先按一定的策略决策一段时间后，再开始探索，则可以节省很多探索预算，有更大的几率探索更重要的部分。

直觉 这就像在迷宫中寻路：如果你一开始就到处乱撞，会浪费大量时间在起点附近；但如果你先沿着一条路径走一段，到达瓶颈处再开始探索，就能更有效地找到突破口。

4.3 蒙特卡洛轨迹树 (MCT2) 规划

该方法以 QMIX 作为基线 (Baseline)，核心创新是引入蒙特卡洛轨迹树 (MCT2) 规划。算法整体架构为：

1. 通过 MCT2 规划，选出当前最优模板路径；
2. 智能体符合模板路线时不探索，跳出模板时开始探索。

4.4 稳定前缀策略

MCT2 的构建过程如下：

1. 使用 EM 算法，将状态 s 聚类成为原型（聚类中心）；
2. 将 replay buffer 中的数据转换为原型转移数据；
3. 根据原型构建蒙特卡洛轨迹树，其枝干仅以原型到原型转换为扩展，而不是以行为进行扩展；

4. 智能体不论采用什么策略，只要其进入到的下一时刻的状态是正确的，即判断其依旧在模板中。

直觉 关键洞察是：我们关心的不是“智能体做了什么具体动作”，而是“智能体到达了什么状态”。只要状态转移符合模板路径，就不需要探索。只有当状态偏离模板时，才开始消耗探索预算。这大大节省了不必要的探索。

为解决**策略偏移**（Policy Shift）的问题，每隔一个周期都摧毁轨迹树，并根据最新经验回放缓存中的数据重新构建 MCTT，在扩展根节点时加入**狄利克雷噪声**（Dirichlet Noise）。

4.5 策略训练与 Q 值聚合

训练过程中：

- 从经验回放缓存中采样轨迹数据，取每条轨迹的初始状态并转换成原型；
- 从 MCTT 中根节点开始规划，为每条轨迹规划出当前最优的模板路径；
- 将真实轨迹转换为真实原型轨迹，与当前最优模板路径对比，找到每条轨迹中不符合模板的部分；
- 将符合模板部分的 $Q_{t,t}$ 相聚合；
- 计算 TD target 值时根据是否符合模板选用聚合的 Q_{t-MIX} 或者原本的 $Q_{t,t}$ ；
- 计算 TD error 后采用 MSE loss 进行训练。

例 假设模板路径为原型 $c_1 \rightarrow c_2 \rightarrow c_3$ ，智能体实际经历了 $c_1 \rightarrow c_2 \rightarrow c_4$ 。前两步（ $c_1 \rightarrow c_2$ ）符合模板，这两步的 Q 值会被聚合到 Q_{t-MIX} 中作为稳定的学习信号。第三步（ $c_2 \rightarrow c_4$ ）跳出了模板，这一步使用原始的 $Q_{t,t}$ 计算，并触发探索以寻找回到模板路径或发现新路径的机会。

5 自适应 TD(λ) 计算方法

5.1 TD(λ) 的平衡难题

在计算 target 的时候，一个核心问题是：应该采用蒙特卡洛采样得到的累积回报，还是采用从已存在的 Q 函数中自举得到新的价值？

TD(λ) 平衡两种更新方式：

- **TD 更新（自举）**：用估计值更新估计值，方差小但有偏差；
- **蒙特卡洛采样（累积回报）**：用真实回报更新，无偏差但方差大。

λ 值域 $[0, 1]$ ，极大影响算法收敛表现。传统方法中 λ 被设定为一个不变的定值超参，且该超参的值选定会极大影响算法的收敛表现。

5.2 相关工作回顾

- **SMIX(λ)** 算法使用离策略进行训练，通过避免贪婪假设来实现稳定的集中价值函数，并将 SMIX(λ) 的概念与 $Q(\lambda)$ 连接起来；
- **ETD(λ)** 通过对 TD(λ) 更新进行适当加权，确保了线性情况下的收敛性；
- **RIIT** 论文讨论了 QMIX 算法的单调性和实现技巧，并在训练过程之前提供了 λ 值建议；

- **重要性采样 (Importance Sampling)** 是校正行为策略与目标策略之间差异的最简单方法，但存在较大的方差。 $Q^*(\lambda)$ 引入了一种基于 Q 基线的偏离策略校正，避免了方差膨胀但不能保证任意行为策略与目标策略的收敛性。

5.3 离策略不稳定的根源

离策略强化学习算法不稳定的根本原因是：行为 μ 策略（旧）与当前 π 策略（新）不同带来的差异。经验回放缓存中的数据是不同时间的策略采到的数据的混合，因此直接使用这些数据进行训练会引入偏差。

5.4 ATD(λ) 方法

ATD(λ) 方法引入了一种自适应 λ 值计算方法，即基于采样的状态转移出现在当前策略的可能性，并在训练过程中确定自适应计算 λ 值，而不是在训练过程之前预设一个超参数。

主体架构为：将智能体与环境交互得到的数据分别加入一个大的 replay buffer 和一个小的 replay buffer 中。大 replay buffer 中存储更多的不同时刻的策略采样得到的轨迹（off-policy），而小 replay buffer 存储的轨迹更贴近当前的策略（on-policy）。

5.5 同策略程度的计算

定义 d_μ 为稳定分布，经验回放缓存 D 中的数据根据该分布采样得到； d_π 为在当前策略下的状态-行为分布。

密度比定义为：

$$\omega(s, a) := \frac{d_{\pi(s, a)}}{d_{\mu(s, a)}} \quad (5.1)$$

当满足 $0 \leq c_t \leq \frac{2d_{\mu(s|a)}}{d_{\pi(s|a)}}$ 条件时，算子 $\mathcal{R}$ 是 γ -收缩映射的。

直觉 密度比 $\omega(s, a)$ 衡量的是“这条经验在当前策略下出现的可能性”与“这条经验在历史行为策略下出现的可能性”之比。如果比值接近 1，说明这条经验仍然与当前策略高度相关，可以用较大的 λ （更信任自举）；如果比值偏离 1，说明这条经验已经过时，应该用较小的 λ （更信任蒙特卡洛采样）。

然而 d_π 很难准确估计，因为 d_π 需要在策略（on-policy）的交互，而交互的资源比较少。同时，在计算 $\omega(s, a)$ 时，因为 replay buffer D 中的数据是不同时间的策略采到的数据的混合，因此比值也很难计算。

解决方案是采用一组较早生成的轨迹与一组较新生成的轨迹之间的 f -散度变分表示来估计密度比。 $\omega(s, a)$ 可以通过一个参数为 φ 的神经网络来拟合。

因为 λ 的值域为 $[0, 1]$ ，因此可以通过 **sigmoid** 激活函数来激活。在本文中采取的 f 为 **Jensen-Shannon 散度** (Jensen-Shannon Divergence)，即 $f(x) = x \log x + (1 - x) \log(1 - x)$ 。则损失函数成为二项交叉熵损失。

最终的 TD 更新目标为：

$$\lambda(s, a) = \gamma \sigma(\omega(s, a)) \quad (5.2)$$

例 假设在某状态-行为对 (s, a) 处，神经网络拟合得到密度比 $\omega(s, a) = 1.5$ ，说明当前策略下该经验的出现概率高于历史策略。取 $\gamma = 0.9$ ，则 $\lambda(s, a) = 0.9 \times \sigma(1.5) \approx 0.9 \times 0.8176 \approx 0.7358$ 。这意味着此处的更新将更偏向 TD 自举（ λ 较大），因为该经验与当前策略高度相关。反之，若 $\omega(s, a) = 0.3$ ，则 $\lambda \approx 0.9 \times \sigma(0.3) \approx 0.9 \times 0.5744 \approx 0.5170$ ，更新将更多依赖蒙特卡洛采样。

5.6 ATD 方法的增强效果

ATD 方法对基于价值的方法和基于策略的方法都有显著的增强效果，验证了自适应 λ 计算的通用性。

6 大模型与强化学习

6.1 为什么要用大模型做决策

多智能体强化学习面临三大挑战：

- 强化学习得到的策略迁移性比较差；
- 可能需要数百万步的训练步数；
- 深度学习模型的可解释性差。

而**大模型决策树**具有独特优势：

- **白盒决策**（White-box Decision）：决策过程可解释；
- 大模型一轮 inference 即可完成决策；
- 决策树自身具备可解释性。

核心问题是：大模型能不能作为**元策略**（Meta-Policy），通过生成决策树策略完成决策任务？

6.2 行为树代码生成：Planner-Coder-Critic

大模型通过三个模块协作生成白盒决策树代码：

1. **Planner**（规划器）：分析任务，制定策略。接收环境提示（Environment Prompt），分析地图信息、单位属性、敌方位置等，提出最重要的战术及使用条件。
2. **Coder**（编码器）：将策略转化为可执行 Python 代码。将战术骨架翻译为决策树脚本（Python Scripts）。
3. **Critic**（批判器）：审查代码执行结果，分析失败原因，提出改进建议。

6.3 实战案例：Stalker 对抗 Spine Crawler

在 SMAC 环境（星际争霸 II）的 2s_vs_1sc 地图中，两个 Stalker 单位需要击败一个 Spine Crawler 结构。Stalker 有 80 生命值、80 护盾、6 攻击距离、4.13 速度、13 伤害和 9.7 DPS；Spine Crawler 有 300 生命值、0 护盾、7 攻击距离、25 伤害和 18.9 DPS。

Planner 提出“Hit and Run”（打带跑）战术：当敌方在攻击范围内时攻击，攻击后撤退以避免反击。

Coder 第一版生成的核心代码如下：

```

class HitAndRunBot(BotAI):
    async def on_step(self, iteration: int):
        stalkers = self.units(UnitTypeId.STALKER)
        spine_crawler = self.enemy_structures.first
        if not spine_crawler or not stalkers.exists:
            return
        for stalker in stalkers:
            if stalker.distance_to(spine_crawler) <= stalker.ground_range + 1:
                stalker.attack(spine_crawler)
            else:
                stalker.move(spine_crawler.position)
            if stalker.weapon_cooldown > 0:
                retreat_position = stalker.position.towards(self.start_location, 3)
                stalker.move(retreat_position)

```

逐行批注：第 3 行获取所有 Stalker 单位；第 4 行获取敌方建筑；第 5-6 行若无目标或无单位则跳过；第 8 行判断 Stalker 是否在攻击范围内（地面攻击距离加 1 的容差）；第 9 行在范围内则攻击；第 11 行不在范围内则靠近敌方；第 12-13 行武器冷却时向起点方向撤退 3 个单位。

第一版执行结果：10 局中赢 4 局输 6 局，胜率 40%，平均得分 227.5，造成伤害 280.0，承受生命值伤害 140.66，承受护盾伤害 170.76。

Critic 分析出四个问题：

- **撤退机制不一致** (Inconsistent Retreat Mechanism)：撤退仅在武器冷却时触发，未考虑血量和护盾；
- **未考虑血量和护盾水平** (No Consideration for Health and Shield Levels)：低血量时仍继续攻击导致损失；
- **固定撤退距离** (Fixed Retreat Distance)：撤退 3 个单位可能不足以脱离敌方攻击范围 (Spine Crawler 攻击距离为 7)；
- **无群体协调** (No Group Coordination)：两个 Stalker 同时攻击或同时撤退，无法分散敌方注意力。

改进后的代码：

```

if stalker.weapon_cooldown > 0 or stalker.shield_percentage < 0.2 or
stalker.health_percentage < 0.2:
    retreat_distance = 5
    retreat_position = stalker.position.towards(self.start_location,
retreat_distance)
    stalker.move(retreat_position)
if len(stalkers) == 2:
    stalker1, stalker2 = stalkers
    if stalker1.weapon_cooldown > 0 and stalker2.weapon_cooldown == 0:
        stalker2.attack(spine_crawler)
    elif stalker2.weapon_cooldown > 0 and stalker1.weapon_cooldown == 0:
        stalker1.attack(spine_crawler)

```

逐行批注：第 1 行将撤退条件从仅武器冷却扩展为武器冷却或护盾低于 20% 或生命值低于 20%；第 2 行将撤退距离从 3 增大到 5，确保脱离 Spine Crawler 的攻击范围；第 5-8 行实现群体协调，当一个 Stalker 撤退时另一个继续攻击，交替输出以分散敌方注意力。

改进后执行结果：10 局全胜，胜率 100%，平均得分 350.0，造成伤害 331.2，承受生命值伤害 115.31，承受护盾伤害 405.09。

例 从 40% 胜率到 100% 胜率，关键改进只有三处：动态撤退条件（考虑血量和护盾）、增大撤退距离（从 3 到 5）、群体协调（交替攻击而非同步）。这说明了 Critic 模块“分析问题、提出改进”的闭环反馈对策略优化的重要性。

6.4 行为树生成能力蒸馏与提升

由于模型太大难以微调，需要将大模型知识蒸馏到小 LLM。代码数据过少时，需要另一个大模型做数据增广。SMAC 作为奖励模型提供代码的真实胜率和分数。训练方法包括 SFT（监督微调）、DPO（直接偏好优化）和 GRPO。

6.5 GRPO 与思考长度

GRPO（Group Relative Policy Optimization）是一种 RL 优化算法。在 SMAC 决策任务上的评分规则为：

- -1 分：无法从大模型的响应中检索到有效行为树；
- -0.5 分：行为树脚本中存在错误；
- 0 至 1 分：生成代码在 SMAC 任务中的胜率。

例 实验发现一个反直觉现象：在 SMAC 决策任务上，胜率随思考内容的减少而增加。在决策问题上，GRPO 的优化方向更倾向于更精确、更短的回答。这与数学推理问题截然相反（后者思考越多越好），说明决策问题需要不同的策略。

6.6 行为树策略辅助强化学习训练

行为树策略还可以辅助强化学习训练，通过两个机制：

- **策略行为对齐**：将行为树生成的策略与强化学习策略对齐；
- **“接力”时间控制退火机制**：控制何时由行为树策略切换到强化学习策略。

例如，当单位生命值低于 30% 时，判断最近敌人方向并执行 SMAC 的动作序列后撤（`unit.move(position)`）。

6.7 SMAC-Hard

SMACV1 存在以下局限：

- 初始化丰富程度不够，相对单一，MARL 算法在这个环境中更像树形展开然后得到一条固定的路径；
- 开环控制已然可以解决大部分的 SMACV1 的场景，即决策仅取决于时间步，状态和观测的信息可以被掩盖；
- 对手是内置脚本，过于简单，存在取巧的获胜方式。

SMACV2 引入了初始化设定，兵种和位置可以通过概率形式生成。SMAC-Hard 进一步提出更大挑战：

- **混合对手脚本（Mixed Opponent Strategy）**：将 LLM 生成的脚本应用为对手策略，避免智能体过拟合到某一个对手策略或利用对手策略的漏洞而拟合到取巧方法上；

- 自博弈接口 (Self-play Interface);
- 战争迷雾 (Fog of War)、技能释放、还原视野、地形优势等。

SMAC-Hard 的核心思想是“对抗出真知”。如果对手太弱或太固定，智能体可能学到的是取巧方法而非真正的策略。通过混合不同强度的对手脚本，可以迫使智能体学到更鲁棒 (Robust) 的策略。

7 并行加速：PaMoRL

7.1 MBRL 的计算代价

基于模型的强化学习表现很好，但代价是额外的计算开销、内存开销和训练时间。在 Atari100K 基准上（大约 2 小时的游玩数据），各方法的训练开销如下：

方法	训练时间	硬件需求
SimPLE	5 days	1 x P100
EfficientZero	7 hours	4 x RTX3090
IRIS	7 days	1 x A100
TWM	10 hours	1 x A100
DreamerV3	12 hours	1 x V100
STORM	9.3 hours	1 x RTX3090

7.2 并行扫描 (Parallel Scan)

如何克服计算开销？可以从大模型领域中 Transformer 模型的挑战者中汲取灵感。核心思想是**并行训练与串行推理**，核心技术是**并行扫描 (Parallel Scan)**。

给定序列 $a_1, a_2, \dots, a_T$ 和 $b_1, b_2, \dots, b_T$ ，状态变量 $x_1, x_2, \dots, x_T$ 的线性动力系统可以通过以下方程计算：

$$x_1 = b_1, \quad x_t = a_t x_{t-1} + b_t, \quad t = 2, \dots, T \quad (7.1)$$

直觉 看起来计算 $x_1, \dots, x_T$ 需要进行 T 步串行计算，因为每个 x_t 都依赖于前一个 x_{t-1} 。但并行扫描巧妙地利用了结合律，将 T 步串行计算降至 $\log_2 T$ 或 $2\log_2 T$ 步并行计算。

例 当 $T = 8$ 时，传统串行计算需要 8 步。而采用不同的并行扫描算法，分别只需要 $\log_2 8 = 3$ 步或 $2\log_2 8 = 6$ 步即可完成。对于 $T = 1000$ ，串行需要 1000 步，而并行扫描仅需约 10 到 20 步，加速约 50 到 100 倍。

7.3 PaMoRL 框架

PaMoRL (Parallelized Model-based RL) 提出了将并行扫描算法同时应用于世界模型学习和策略学习（资格迹估计）的框架。该框架包含三个并行化模块：

- **World Model Learning** (世界模型学习，并行)：使用并行扫描替代串行的循环计算；
- **Imagination** (想象，循环)：在学到的世界模型上进行策略推演；

- **Eligibility Trace Estimation** (资格迹估计, 并行): TD- λ 的资格迹计算也使用并行扫描加速。

7.4 Linear Attention 变体

针对强化学习的任务特性, PaMoRL 在世界模型中使用了一种 **Linear Attention** (线性注意力) 的变体结构, 包含以下关键组件:

- **Token Mixing** 模块: 替代标准注意力机制, 降低计算复杂度;
- **Data-dependent Decay Rate** (数据依赖的衰减率, 即遗忘门): 保证训练稳定性;
- **Post-Norm**: 提升表达能力的同时保持最小复杂度。

标准 *Transformer* 的注意力机制复杂度为 $O(T^2)$, 而 *Linear Attention* 通过核函数近似将其降至 $O(T)$, 再配合并行扫描, 实现了线性时间内的序列建模。

7.5 PaMoRL 的效果

PaMoRL 同时实现了 MBRL 级别的样本效率与 MFRL 级别的计算效率。它带来了可接受的额外内存开销, 但能够换来显著的计算加速。

关键稳定性组件的重要性也得到了验证:

- **RMSNorm**、**Token Mixing** 和 **Data-dependent Decay Rate** 对强化学习任务的训练稳定性很重要;
- **BatchNorm Trick** 对于提取图像输入中的细节信息具有非常重要的作用。

8 具身智能

8.1 核心问题

具身智能 (Embodied Intelligence) 面临的两个核心问题是:

- **采样效率低**: 强化学习的样本量一般会达到上千万, 在现实环境中很难采集如此数量级的样本;
- **安全问题**: 强化学习需要大量试错, 该特性可能会损伤机器人自身, 也会对周围环境甚至生物造成伤害。

直觉 想象一个机器人学走路。如果直接在现实世界中用强化学习训练, 它需要摔倒几百万次才能学会, 这期间机器人会被摔坏无数次。因此我们需要在仿真环境中训练, 再把策略迁移到现实世界。

8.2 仿真平台

为了解决采样效率和安全问题, 研究者开发了多种仿真平台:

- **MuJoCo** (Multi-Joint dynamics with Contact, 带接触的多关节动力学): 作为一款通用物理引擎, 致力于推动机器人学、生物力学、图形动画、机器学习等领域的研究与开发, 这些领域都需要对关节结构与环境的交互进行快速精确的模拟。

- **Isaac Gym**: NVIDIA 为强化学习打造的物理模拟环境，支持基于 GPU 的大规模并行物理仿真，支持导入多种机器人描述文件，并自动进行凸分解以用于物理模拟，支持多种环境传感器、多种物理参数的域随机化。
- **Isaac Lab**: 基于 NVIDIA Isaac Sim 构建的统一且模块化的机器人学习框架，旨在简化机器人研究中的常见工作流程，能够利用最新仿真技术实现逼真场景渲染及高效快速仿真。
- **SAPIEN**: 一个高度拟真、物理特性丰富的仿真环境，内含大规模铰接物体数据集，主要面向精细操作领域，支持多种需要精细部件级理解的各类机器人视觉与交互任务。

8.3 Real2Sim / Sim2Real 闭环

为了弥合仿真与现实的差距，研究者提出了多种迁移范式：

- **Real2Sim**: 将真实世界数据导入仿真环境；
- **Sim2Real**: 在仿真中训练策略后迁移回真实世界；
- **Sim2Sim**: 仿真环境之间的迁移；
- **Real2Real**: 真实世界之间的直接迁移。

Real2Sim / Sim2Real 闭环的核心挑战是**现实差距 (Reality Gap)**：仿真环境的物理参数与现实世界总存在差异。域随机化 (*Domain Randomization*) 是常用的缓解手段，即在训练时随机化物理参数，使策略对参数变化具有鲁棒性。

8.4 UniHSI

UniHSI 是首个由大语言模型驱动的统一交互框架。它将场景交互建模为**接触链 (Contact Chain)**，用人体与物体部分的接触来描述交互行为。

UniHSI 包含两个核心模块：

- **上游大语言模型规划器 (Upstream LLM Planner)**: 将语言输入转化为交互接触链形式的任务计划；
- **下游人型机器人强化学习运动控制策略 (Downstream RL Locomotion Controller)**: 执行交互接触链。

UniHSI 支持长程交互、多物品交互以及同一物品的多样化交互，还支持基于大语言模型的“多智能体”交互。

例 当输入“走到桌子旁，拿起杯子，递给另一个人”这样的语言指令时，上游 LLM 规划器将其分解为一系列接触链节点（人体部位与物体部位的接触关系），下游 RL 控制器再逐个执行这些节点对应的运动控制策略。

9 本章你将学会

1. 理解强化学习从“被动学习知识”到“主动积累经验”的范式转变，以及 Sutton“经验时代”的核心论点。
2. 掌握 MBRL 与 MFRL 的权衡关系，理解自举误差在 MDP 树中累积放大的机制，以及“先监督后强化”接力架构的设计动机。
3. 能够解释 $TD(\lambda)$ 平衡 TD 自举与蒙特卡洛采样的原理，理解 $ATD(\lambda)$ 通过密度比自适应计算 λ 的方法。

- 理解大模型行为树代码生成的 Planner-Coder-Critic 三模块协作流程，以及 GRPO 在决策任务上“思考越少越好”的反直觉现象。
- 掌握并行扫描将串行 T 步计算降至 $\log_2 T$ 步的原理，以及 PaMoRL 同时实现高样本效率与高计算效率的设计思路。

10 要点速查

要点	说明
经验时代	AI 需通过与环境交互产生的经验来学习，超越人类数据
MBRL vs MFRL	MBRL 高效有累积误差，MFRL 低效但渐近最佳
自举误差	随 MDP 树回传累积放大，智能体越多误差越大
接力架构	先监督学习给出好起点，再用 RL 精调
上下文预测模型	同时预测动作（分类）和下一观测（回归），预测不准时交由 RL 接手
MCT2 探索	符合模板不探索，跳出模板才探索，节省探索预算
稳定前缀策略	以原型到原型转换扩展树，不关心具体动作只关心状态
TD(λ)	平衡 TD 自举（低方差有偏差）与 MC 采样（无偏差高方差）
ATD(λ)	通过密度比 $\omega(s, a)$ 自适应计算 $\lambda = \gamma\sigma(\omega)$
离策略不稳定	根源是行为策略 μ 与当前策略 π 不同
Planner-Coder-Critic	大模型三模块协作生成白盒决策树代码
决策 vs 数学	决策问题胜率随思考减少而增加，与数学推理相反
SMAC-Hard	混合对手脚本，避免过拟合到固定对手
Parallel Scan	串行 T 步降至 $\log_2 T$ 或 $2\log_2 T$ 步
PaMoRL	MBRL 级样本效率 + MFRL 级计算效率
PaMoRL 稳定性	RMSNorm + Token Mixing + Data-dependent Decay Rate + BatchNorm Trick
具身智能	Real2Sim/Sim2Real 闭环解决仿真到现实迁移
UniHSI	LLM 驱动的统一交互框架，将交互建模为接触链

11 小结

强化学习是通向“经验时代”的关键路径，从被动学习到主动积累经验。本章覆盖了六个核心主题：基于经验缓存样本监督的模型接力方法（自举误差、上下文预测模型、蒙特卡洛轨迹树）、基于历史样本建模的多智能博弈探索（MCT2 规划、稳定前缀策略）、自适应 TD(λ) 计算方法（密度比估计、ATD 自适应 λ ）、行为树代码生成策略优化（Planner-Coder-Critic、GRPO、SMAC-Hard）、并行化基于模型的强化学习（并行扫描、PaMoRL），以及具身智能（仿真平台、Real2Sim/Sim2Real、UniHSI）。

核心洞察有三：第一，决策问题与数学推理问题需要不同的策略，“先监督后强化”的接力架构是解决自举误差累积的有效方案；第二，大模型可以作为元策略生成可解释的白盒决策树代码，且决策任务中更精确简短的回答反而更有效；第三，并行扫描技术可以在不牺牲样本效率的前提下大幅提升计算效率，是连接 MBRL 与 MFRL 优势的桥梁。

讲义基于 HPC101-2025 Day11「机器学习高级话题」课程内容编写